

VIP C2 PROGRAMME — FULL STACK DEVELOPMENT WITH MERN

Full Stack Development (FSD) Documentation

MedConnect — Doctor Appointment Booking Platform

Phase: Final Academic Submission

Document Version: 1.0

Date: 16th of June, 2026

Prepared by: Kolan

1. Introduction

Project Title

MedConnect — A Production-Grade Full-Stack MERN Doctor Appointment Booking & Telemedicine Platform.

Team Members

- **Kolan** — Lead Full-Stack Developer & Solutions Architect
 - *Role:* Decoupled REST API engineering, role-based JWT validations, schema design, React dashboard components, Socket.io real-time triggers integration, and local fallback configurations.

2. Project Overview

Purpose

MedConnect is engineered to address scheduling friction, record safety, and verification opacity in healthcare discovery. For patients, it provides an intuitive discovery search directory with filtering parameters, real-time booking availability, and secure clinical records locker. For medical practitioners, it offers a workspace to onboard credentials, configure calendar hours, audit patient medical histories, and manage consultation workflow requests. For administrators, it offers dashboard toggles to review registrations, moderate database logs, and inspect platform metrics.

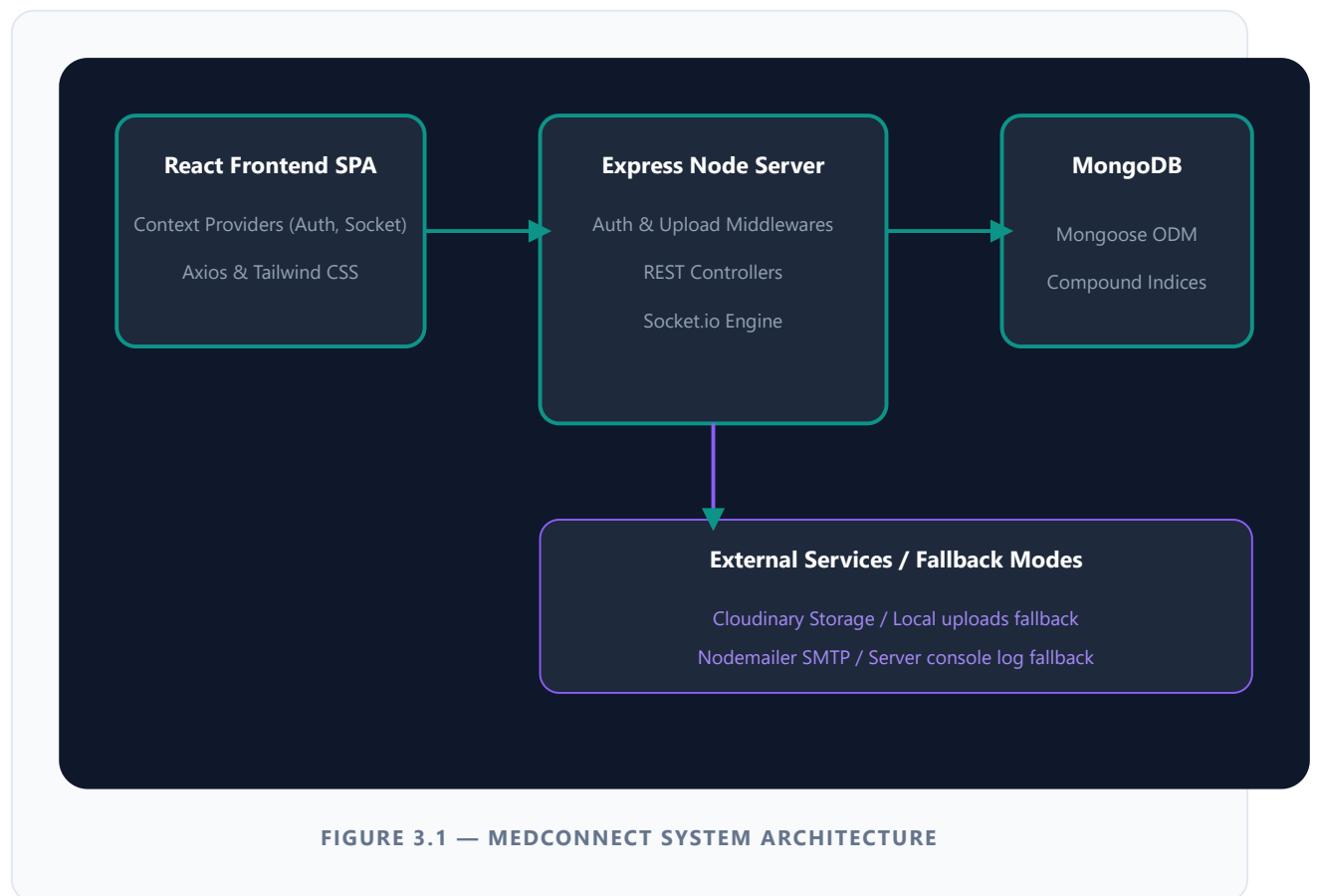
Core Application Features

- **User Identity & Verification:** JWT credentials authentication mapping custom onboarding roles (Patient, Doctor, Admin) with email confirmation codes fallbacks.
- **Specialist Discovery Directory:** Instant doctors lookup engine filtering search matches by specialty, clinic location, experience years, fee ranges, and sorting preferences.
- **Anti-Collision Scheduling:** Appointment booking calendar with compound unique index constraints in Mongoose to prevent double-booking slot conflicts.
- **HIPAA-Safe Records Sharing:** Secure patient PDF/image uploads, restricted strictly to consulting doctors with active appointment history logs.
- **Socket.io Notifications:** Real-time push updates displaying browser alerts upon confirmations, completes, or cancellation events.
- **Aggregate Feedback Analytics:** Automatic re-calculation of average ratings and total reviews counts on the Doctor document on patient feedback submissions.

3. Architecture

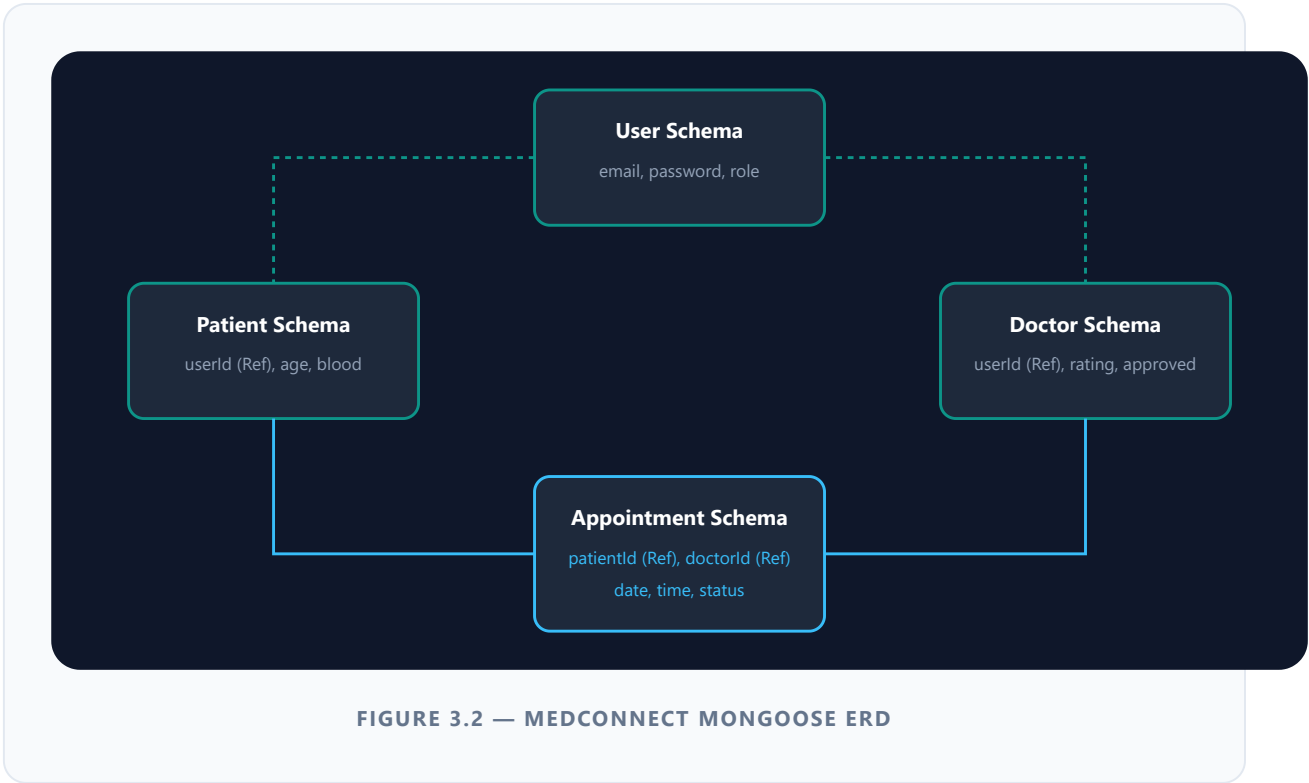
System Architecture Details

The architecture utilizes a completely decoupled Client-Server model. The client React SPA interacts with the Node.js/Express REST server utilizing Axios JSON payloads. Database connections are handled via Mongoose ODM to a MongoDB cluster.



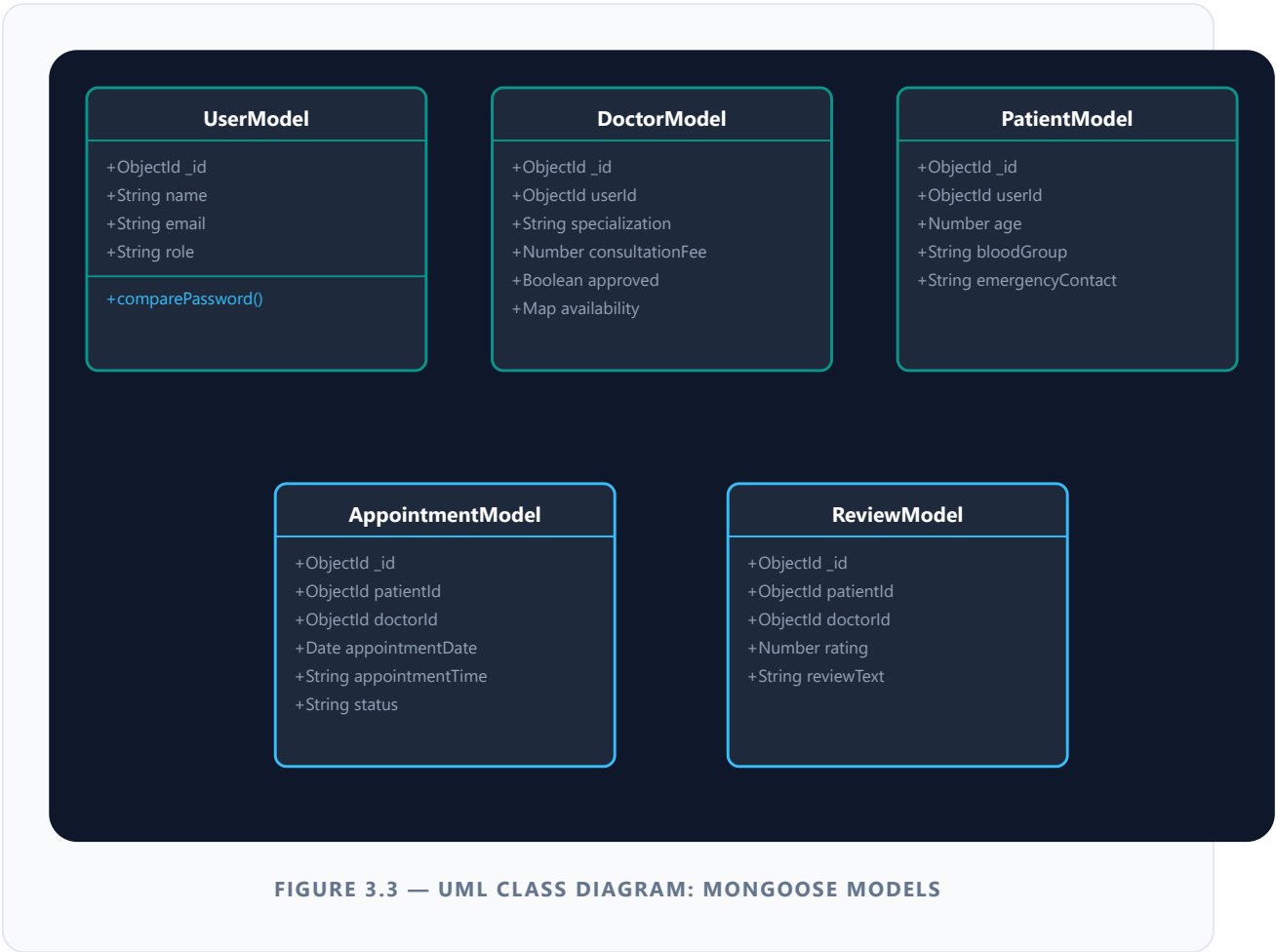
Entity Relationship Diagram

The relational integrity among Mongoose document references is visualised below. User document IDs are mapped to Patient or Doctor profile tables depending on custom roles.



UML Class Diagram: Core Mongoose Models

The class diagram below documents the schema properties, relationships, and methods defined in the database model layer.



4. Setup Instructions

Prerequisites

- Node.js v16 or above (Recommended: v18 LTS)
- MongoDB Community Server locally running on port 27017

Installation

1. **Configure environmental secrets:** Create a `.env` file under the server directory:

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/medconnect
JWT_SECRET=medconnect_jwt_super_secret_key_123456
CLOUDINARY_NAME=
CLOUDINARY_KEY=
CLOUDINARY_SECRET=
EMAIL_USER=
EMAIL_PASS=
```

2. **Install backend server dependencies:**

```
cd backend
$env:PATH = "C:\Program Files\nodejs;" + $env:PATH
npm install
```

3. **Install frontend client dependencies:**

```
cd ../frontend
$env:PATH = "C:\Program Files\nodejs;" + $env:PATH
npm install
```

5. Folder Structure

Client-Side layout

```
frontend/  
├─ index.html  
├─ vite.config.js  
└─ src/  
    ├─ App.jsx           <-- Context mounts & Route lists  
    ├─ index.css         <-- Global styles & glassmorphic themes  
    ├─ context/          <-- State providers (Auth, Socket)  
    ├─ layouts/          <-- Shared Header Nav & Sidebars  
    ├─ components/       <-- Timelines, Skeletons, EmptyStates  
    ├─ services/         <-- Axios API interfaces  
    └─ pages/            <-- Authentication & Portals pages
```

Server-Side layout

```
backend/  
├─ config/              <-- Mongoose DB connection pools  
├─ models/              <-- Schemas validation constraints  
├─ middleware/          <-- JWT protecting & upload configs  
├─ controllers/         <-- Business logic handlers  
├─ routes/              <-- Express routes mappings  
├─ services/            <-- Socket pushes & files handlers  
├─ uploads/             <-- Temp local file storage  
└─ server.js            <-- Application main entry file
```

6. Running the Application

To run MedConnect in local development mode, execute both the backend and frontend servers in separate processes:

1. Start Backend API Server

```
cd backend
node server.js
# Output: MedConnect Server is executing in development mode on port 5000
```

2. Start Frontend Dev Client

```
cd frontend
npm run dev
# Output: VITE ready in local: http://localhost:3000
```

7. API Documentation

Method	Endpoint	Auth Required	Description
POST	/api/auth/register	No	Registers user and instantiates Patient/Doctor row references.
POST	/api/auth/login	No	Validates credentials, returns JWT payload token and user data.
PUT	/api/auth/profile	Yes	Updates user names, phone numbers, and sub-profile properties.
GET	/api/doctors	No	Retrieves lists of approved doctors matching search filters.
PUT	/api/doctors/:id/approve	Yes (Admin)	Approves or disapproves doctor registration verification details.
POST	/api/appointments	Yes (Patient)	Schedules a new appointment. Prevents colliding slot conflicts.
PUT	/api/appointments/:id	Yes	Updates status stage tracking (Confirmed, Completed, Cancelled).
POST	/api/reports/upload	Yes (Patient)	Uploads medical records securely (PDF, PNG, JPG).
POST	/api/reviews	Yes (Patient)	Saves feedback reviews. Patient must have completed consultation.
GET	/api/analytics/admin	Yes (Admin)	Generates stats counts and charts metrics.

8. Authentication

MedConnect uses stateless JSON Web Token (JWT) credentials mapping. The token is generated during logins/registrations and signed with a JWT secret key, then forwarded via header Authorization Bearer tokens.

```
export const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
    token = req.headers.authorization.split(' ')[1];
  }
  if (!token) return res.status(401).json({ message: 'Not authorized, token missing' });
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = await User.findById(decoded.id).select('-password');
    next();
  } catch (error) {
    return res.status(401).json({ message: 'Token invalid' });
  }
};
```

9. User Interface

Below is the visual homepage and discovery panel layout running locally on port 3000:

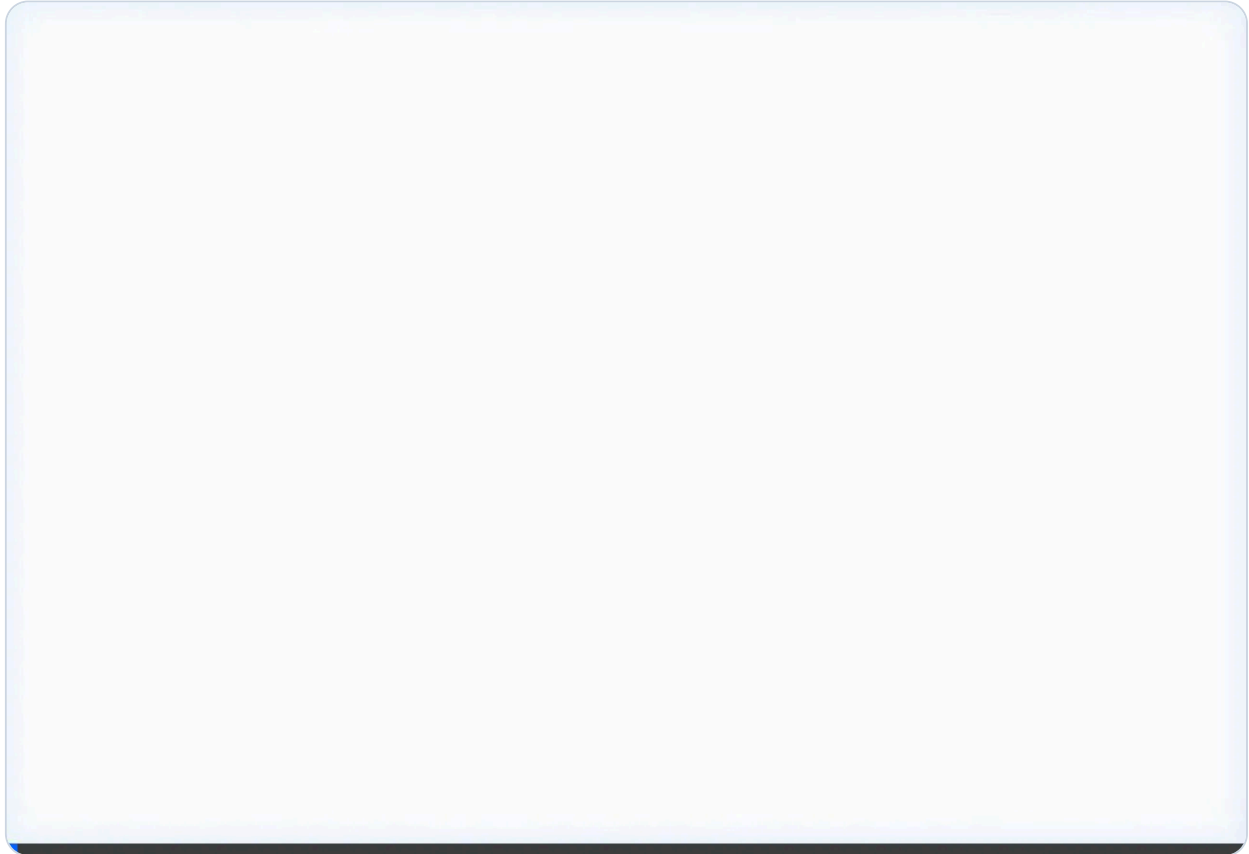


FIGURE 9.1 — MEDCONNECT HOMEPAGE & CATEGORY DISCOVERY UI

10. Testing

Manual User Acceptance Testing (UAT)

We executed the following manual UAT scripts to verify the complete full-stack workflow:

Test ID	Flow Area	Input Action	Expected Outcome	Status
UAT-01	User Registration	Fill out details, choose 'Patient' role, submit.	Account created, patient schema populated, verification email logged.	PASSED
UAT-02	Doctor Onboarding	Register Doctor, fill clinic hours and fees, submit.	Profile saved. Verification status set to pending onboarding approval.	PASSED
UAT-03	Admin Approval	Log in as Admin, navigate to approvals, click 'Approve Verify'.	Doctor status set to approved, visible in public directories.	PASSED
UAT-04	Booking & Timeline	Log in as Patient, pick available slot on Doctor, book.	Socket alert raised on Doctor, slot saved, timeline set to Pending.	PASSED
UAT-05	Double Booking Block	Try booking the exact same date and time slot for the same doctor.	Collision error raised, index constraints prevent write.	PASSED
UAT-06	Record Upload	Patient uploads a PDF medical report file.	File uploaded to uploads/ directory, database reference created.	PASSED
UAT-07	HIPAA Record Shield	Try viewing patient records from a doctor with no appointment history.	Access Denied block raised.	PASSED

11. Demo Credentials

Role	Email Address	Password	Access Area
Patient Portal	patient@medconnect.local	password123	http://localhost:3000/patient-dashboard
Doctor Portal	doctor@medconnect.local	password123	http://localhost:3000/doctor-dashboard
Admin Panel	admin@medconnect.local	password123	http://localhost:3000/admin-dashboard

12. Known Issues

Issue ID	Description	Mitigation Plan
KI-01	Nodemailer log output fallbacks in dev mode.	Integrate production mail server (SendGrid / AWS SES) in .env properties.
KI-02	Local uploads directory storage limits.	Configure Cloudinary API keys to automatically upload files to remote CDN.

13. Future Enhancements

Priority	Enhancement Name	Technical Goal
High	WebRTC Teleconsultations	Embed direct video calls inside doctor/patient dashboard pages.
Medium	Prescription Generator	Provide doctors with formatted editors to compose and sign PDF prescriptions.
Low	Payment Gateways	Integrate Razorpay or Stripe to process fees prior to scheduling.